

BÀI 9: ÔN TẬP TỔNG HỢP

Mục tiêu:

- **Về kiến thức:** Tổng hợp đánh giá kết hợp kết quả các nội dung đã học
- **Về kỹ năng:** Vận dụng thực hành đầy đủ quy trình kiểm thử phần mềm

9.1 Câu hỏi ôn tập lý thuyết

- Câu 1. Các định nghĩa và thuật ngữ cơ bản về kiểm thử phần mềm?
- Câu 2. Phân loại lỗi?
- Câu 3. Phân biệt các mức kiểm thử?
- Câu 4. Trình bày các nguyên tắc kiểm thử?
- Câu 5. So sánh kiểm thử hộp trắng và kiểm thử hộp đen?
- Câu 6. Tại sao kiểm thử hộp trắng lại tốn chi phí cao hơn kiểm thử hộp đen?
- Câu 7. Quy trình kiểm thử dựa trên độ đo?
- Câu 8. Phân biệt các độ đo kiểm thử dòng điều khiển?
- Câu 9. Trình bày các vấn đề phổ biến về dòng dữ liệu?
- Câu 10. Trình bày các khái niệm cơ bản về dòng dữ liệu?
- Câu 11. So sánh kiểm thử dòng điều khiển và kiểm thử dòng dữ liệu?
- Câu 12. Trình bày nguyên tắc chọn dữ liệu thử?
- Câu 13. Trình bày nguyên tắc phân hoạch lớp tương đương?
- Câu 14. Trình bày các thành phần của bảng quyết định?
- Câu 15. Các bước thực hiện kiểm thử giá trị biên?
- Câu 16. Các bước thực hiện kiểm thử lớp tương đương?
- Câu 17. Các bước thực hiện kiểm thử bằng bảng quyết định?
- Câu 18. Các bước thực hiện kiểm thử tổ hợp?
- Câu 19. Nêu quy trình kiểm thử phần mềm?
- Câu 20. Các bước lập kế hoạch kiểm thử?
- Câu 21. Nêu quy trình kiểm thử phần mềm?

9.2 Hệ thống bài tập thực hành

Bài thực hành số 1

1. Mục tiêu kiến thức

- Giúp sinh viên làm quen với việc lập kế hoạch kiểm thử
- Bước đầu thiết kế các testcase chuẩn bị cho kiểm thử chức năng

2. Yêu cầu:

- + Yêu cầu về điều kiện thực hành: máy tính, công cụ kiểm thử tự động
- + Yêu cầu sinh viên: nắm được các bước lập kế hoạch kiểm thử, biết cách thiết kế testcase, biết sử dụng công cụ kiểm thử tự động để thực thi testcase,...

3. Nội dung

Bài 1: Thiết kế testcase cho chức năng đăng nhập.

- Bước 1:
 - XD điều kiện: Gồm 2 điều kiện
 - Điều kiện 1: Nhập User
 - Điều kiện 2: Nhập Password
 - Giá trị của điều kiện: 3 giá trị: đúng (T), sai (F), để trống (B)
 - Hành động của hệ thống: Đăng nhập thành công (T) hay không (F)
- Bước 2:

Đk	TH1	Th2	TH3	TH4	TH5	TH6	TH7	TH8	TH9
Nhập user	T	T	T	F	F	F	B	B	B
Nhập Password	T	F	B	T	F	B	T	F	B

- Bước 3, Bước 4

	Các trường hợp				
Điều kiện	TH1	TH2	TH3	TH4	TH5
Nhập user	T	T	T	F	B
Nhập pass	T	F	B	-	-
Hành động của hệ thống					
Đăng nhập thành công hay không	T	F	F	F	F

- Bước 5:

Id	Tiêu đề	Mô tả kịch bản	Kết quả mong đợi		
TC1	Đăng nhập thành công	1. Mở chức năng đăng nhập 2. Nhập user đúng 3. Nhập password đúng 4. ấn phím đăng nhập	Đăng nhập thành công		
TC2	Đăng nhập không thành công	1. Mở chức năng đăng nhập 2. Nhập user đúng 3. Nhập password sai. 4. Ấn phím đăng nhập	Hệ thống thông báo lỗi sai password		
TC3	Đăng nhập không thành công	1. Mở chức năng đăng nhập 2. Nhập user đúng 3. Không nhập password	Hệ thống thông báo lỗi		

		4. Ấn phím đăng nhập	để trống password		
TC4	Đăng nhập không thành công	1. Mở chức năng đăng nhập 2. Nhập user sai 3. Ấn phím đăng nhập	Hệ thống thông báo lỗi nhập sai user		
TC5	Đăng nhập không thành công	1. Mở chức năng đăng nhập 2. Bỏ trống user 3. Ấn phím đăng nhập	Hệ thống thông báo lỗi bỏ trống user		

Bài 2: Thiết kế testcase cho chức năng cập nhật giỏ hàng.

- Bảng quyết định chức năng cập nhật giỏ hàng

Điều kiện	TH1	TH2	TH3	TH4
Màu sắc	T	B	T	B
Size	T	T	B	B
Hành động				
Kết quả	T	F	F	F

- Kịch bản test chức năng cập nhật giỏ hàng

ID-TC	Tiêu đề	Kịch bản	EO	RO	Kết luận
1	Cập nhật giỏ hàng thành công	- Nhấn vào nút “chọn màu” để thay đổi - Nhấn vào nút “chọn size” để thay đổi - Nhấn nút “cập nhật giỏ hàng”	Hệ thống cho biết cập nhật giỏ hàng thành công		
2	Chưa cập nhật được vào giỏ hàng	- Nhấn vào nút “chọn màu” để thay đổi - Nút “chọn size” bỏ trống - Nhấn nút “cập nhật giỏ hàng”	Hệ thống hiển thị thông báo "Bạn vui lòng chọn size"		
3	Chưa cập nhật được vào giỏ hàng	- Nút “chọn màu” bỏ trống - Nhấn nút “chọn size” để thay đổi - Nhấn nút “cập nhật giỏ hàng”	Hệ thống hiển thị thông báo "Bạn vui lòng chọn màu"		

4	Chưa cập nhật được vào giỏ hàng	- Nút “chọn màu” bỏ trống - Nút “chọn size” bỏ trống - Nhấn nút “cập nhật giỏ hàng”	Hệ thống hiển thị thông báo "Bạn vui lòng chọn màu và size"		
---	---------------------------------	---	---	--	--

ID-TC	Tiêu đề	Kịch bản	EO	RO	Kết luận
1	Cập nhật giỏ hàng thành công	Màu: đen -> Màu: trắng Size: 26 -> Size:25	Hệ thống cho biết cập nhật giỏ hàng thành công	Hệ thống cho biết cập nhật giỏ hàng thành công	Pass
2	Chưa cập nhật được vào giỏ hàng	Màu: đen -> Màu: trắng Size: 26 -> Size:	Hệ thống hiển thị thông báo "Bạn vui lòng chọn size"	Hệ thống hiển thị thông báo "Bạn vui lòng chọn size"	Pass
	Chưa cập nhật được vào giỏ hàng	Màu: đen -> Màu: Size: 26 -> Size:25	Hệ thống hiển thị thông báo "Bạn vui lòng chọn màu"	Hệ thống hiển thị thông báo "Bạn vui lòng chọn màu"	Pass
	Chưa cập nhật được vào giỏ hàng	Màu: đen -> Màu: Size: 26 -> Size:	Hệ thống hiển thị thông báo "Bạn vui lòng chọn size và màu"	Hệ thống hiển thị thông báo "Bạn vui lòng chọn size và màu"	Pass

Bài 3. Xây dựng bảng quyết định cho PreviousDate, hàm ngược của bài toán NextDate.

Bài 4. Xây dựng bảng quyết định cho bài toán giải phương trình bậc hai.

Bài 5. Để có thể rút được tiền từ máy ATM, người dùng cần một trong hai điều kiện: tiền trong tài khoản vẫn còn và lớn hơn số tiền muốn rút (Đối với thẻ debit) hoặc là: người dùng được cấp cho một khoản tín dụng từ trước (Đối với thẻ Credit).

Yêu cầu: Thiết kế testcase kiểm thử chức năng Rút tiền từ máy ATM.

Bài thực hành số 2

1. Mục tiêu:

- + Vẽ đồ thị dòng điều khiển cho đoạn mã nguồn cần kiểm thử
- + Biết cách tính độ phức tạp của đồ thị
- + Liệt kê đủ số đường thi hành tuyến tính độc lập
- + Sinh testcase đảm bảo các độ đo kiểm thử

2. Yêu cầu:

- + Yêu cầu về điều kiện thực hành: máy PC (laptop), công cụ kiểm thử tự động
- + Yêu cầu sinh viên: Thiết kế tescae đảm bảo các độ đo, thực thi kiểm thử với công cụ

3. Nội dung

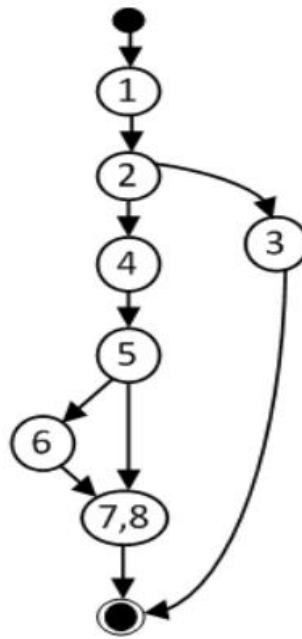
Bài 1. Cho đoạn chương trình sau:

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Hướng dẫn:

1. Vẽ đồ thị:



2. Tính $C = P + 1 = 3$

3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Độ đo C1:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		

Độ đo C2:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		
tc3	1; 2; 4; 5; 7,8	2,3,4,5	lỗi chia cho 0		

Độ đo 3:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5c1; 6; 7,8	0, 2, 3, 5	0		
tc2	1; 2; 4; 5c1; 5c2; 6; 7,8	2, 2, 2, 7	1		
tc3	1; 2; 4; 5c1; 5c2; 7,8	2,3,4,5	lỗi chia cho 0		
tc4	1; 2; 3	2,3,4,4	1		

Bài 2. Cho hàm được viết bằng ngôn ngữ C như sau:

```
//tra ve 1 neu year la nam nhuan, nguoc lai tra ve 0
int LaNamNhuan(int year){
    if( (year%400==0) || (year%4==0 && year%100!=0))
        return 1;

    return 0;
}
```

1. Hãy xây dựng đồ thị dòng điều khiển cho hàm LaNamNhuan
2. Hãy sinh các đường đi và các ca kiểm thử với độ đo C1.
3. Hãy sinh các đường đi và các ca kiểm thử với độ đo C2.
4. Hãy xây dựng đồ thị dòng điều khiển cho hàm LaNamNhuan ứng với độ đo C3.
5. Hãy sinh các đường đi và các ca kiểm thử với độ đo C3.

Bài 3. Cho hàm được viết bằng ngôn ngữ C như sau:

```
//tra lai so ky tu trong xau
int SoKyTu(char a[]){
    int i;
    int total = 0;
    int len = strlen(a); //lay do dai cua xau a
    for (i = 0; i<len; i++){
        if(((a[i]>='A')&&(a[i]<='Z'))||((a[i]>='a')&&(a[i]<='z'))))
            total ++;
    }//end for
    return total;
}
```

1. Hãy xây dựng đồ thị dòng điều khiển cho hàm SoKyTu ứng với độ đo C1 và C2.
2. Hãy sinh các đường đi và các ca kiểm thử với độ đo C1.
3. Hãy sinh các đường đi và các ca kiểm thử với độ đo C2.
4. Hãy sinh các đường đi và các ca kiểm thử với độ đo C3.
5. Hãy sinh các ca kiểm thử để kiểm thử vòng lặp for.

Bài thực hành số 3

1. Mục tiêu:

- + Vẽ đồ thị dòng điều khiển cho đoạn mã nguồn cần kiểm thử
- + Biết cách tính độ phức tạp của đồ thị
- + Liệt kê đủ số đường thi hành tuyến tính độc lập
- + Sinh testcase đảm bảo các độ đo kiểm thử

2. Yêu cầu:

- + Yêu cầu về điều kiện thực hành: máy PC (laptop), công cụ kiểm thử tự động
- + Yêu cầu sinh viên: Thiết kế tescae đảm bảo các độ đo, thực thi kiểm thử với công cụ

3. Nội dung

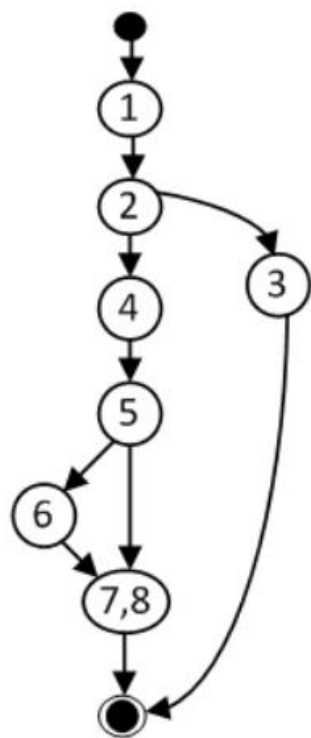
Bài 1. Cho đoạn chương trình sau:

```
float foo(int a, int b, int c, int d){  
1.  float e;  
2.  if (a==0)  
3.      return 0;  
4.  int x = 0;  
5.  if ((a==b) || (c==d))  
6.      x = 1;  
7.  e = 1/x;  
8.  return e;  
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Hướng dẫn:

1. Vẽ đồ thị:



2. Tính $C = P + 1 = 3$

3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Độ đo C1:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		

Độ đo C2:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5; 6; 7,8	2, 2, 3, 5	1		
tc2	1; 2; 3	0, 3, 2, 7	0		
tc3	1; 2; 4; 5; 7,8	2,3,4,5	lỗi chia cho 0		

Độ đo 3:

ID	Test Path	Inputs	EO	RO	Note
tc1	1; 2; 4; 5c1; 6; 7,8	0, 2, 3, 5	0		
tc2	1; 2; 4; 5c1; 5c2; 6; 7,8	2, 2, 2, 7	1		
tc3	1; 2; 4; 5c1; 5c2; 7,8	2,3,4,5	lỗi chia cho 0		
tc4	1; 2; 3	2,3,4,4	1		

Bài 2. Cho đoạn chương trình sau:

```
double    Power(double x, int n)
{
    double    result = 1;
    for(int i=1; i< n; i++)
        result *= x;

    return result;
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3

Bài 3. Cho đoạn chương trình sau:

```
int sum(int a[], int n){
    int i;
    int total = 0;
    for(i=0; i<n; i++)
        total = total + a[i];
    return total;
}
```

1. Vẽ đồ thị dòng điều khiển cho đoạn chương trình trên
2. Tính hằng số cyclomatic (kí hiệu C) cho sơ đồ vừa vẽ?
3. Sinh các đường đi và các testcase đảm bảo độ đo C1, C2, C3
4. Sinh các đường đi và các testcase đảm bảo độ đo cho vòng lặp

Bài 4. Hãy sinh các ca kiểm thử để kiểm thử các vòng lặp lồng nhau của hàm SelectionSort

```
void SelectionSort(int a[], int size){
    int I,j;
    for(i=0; i<size-1; i++){
        int min = I;
        for(j=i+1; j<size; j++)
            if(a[j]<a[min])
                min = j;
        int tem = a[i];
        a[i] = a[min];
        a[min] = tem;
    }//end for
}//the end
```

Bài thực hành số 4

1. Mục tiêu:

- + Vẽ đồ thị dòng dữ liệu cho đoạn mã nguồn cần kiểm thử
- + Xác định tất cả các *Complete-path* từ đồ thị
- + Xác định tất cả các *Def-clear-path* của các biến
- + Xác định các *All-p-uses/Some-c-uses* và *All-c-uses/Some-p-uses*

2. Yêu cầu:

- + Yêu cầu về điều kiện thực hành: máy PC (laptop), công cụ kiểm thử tự động
- + Yêu cầu sinh viên: Thiết kế testcases đảm bảo các độ đo, thực thi kiểm thử với công cụ

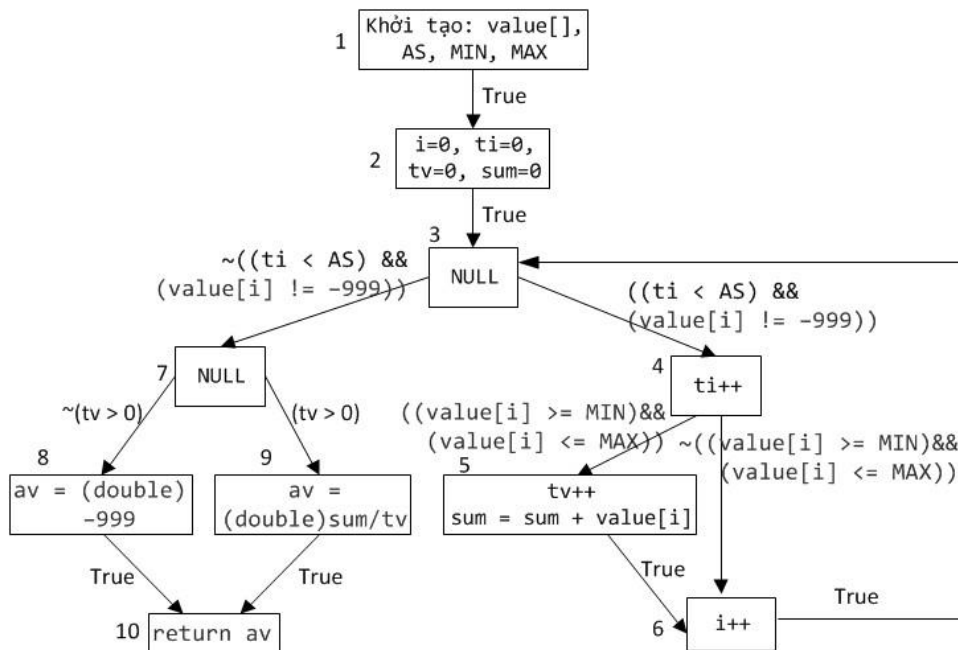
Bài 1. Cho đoạn mã nguồn như sau:

```
double ReturnAverage(int value[], int AS, int MIN, int MAX){
    int i, ti, tv, sum;
    double av;
    i = 0; ti = 0; tv = 0; sum = 0;
    while (ti < AS && value[i] != -999) {
        ti++;
        if (value[i] >= MIN && value[i] <= MAX) {
            tv++;
            sum = sum + value[i];
        }
        i++;
    } //end while
    if (tv > 0)
        av = (double)sum/tv;
    else
        av = (double) -999;
    return (av);
} //the end
```

1. Vẽ đồ thị dòng dữ liệu cho đoạn mã nguồn trên
2. Xác định các def và c-use của các đỉnh
3. Xác định các Complete-path từ đồ thị dòng dữ liệu

Hướng dẫn:

1. Vẽ đồ thị dòng dữ liệu:



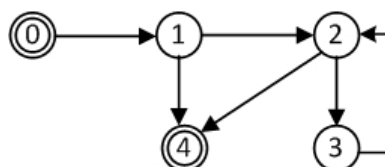
2. Xác định các def và c-use của các đỉnh

Đỉnh i	$def(i)$	$c-use(i)$
1	{value, AS, MIN, MAX}	{}
2	{i, ti, tv, sum}	{}
3	{}	{}
4	{ti}	{ti}
5	{tv, sum}	{tv, i, sum, value}
6	{i}	{i}
7	{}	{}
8	{av}	{}
9	{av}	{sum, tv}
10	{}	{av}

3. Xác định các Complete-path từ đồ thị dòng dữ liệu

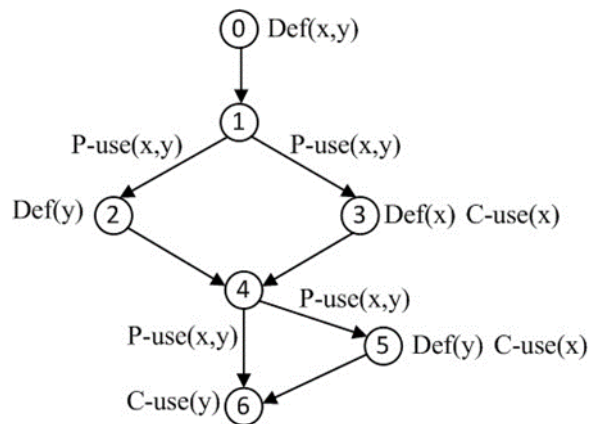
- (1 - 2 - 3 - 4 - 5 - 6 - 3 - 7 - 8 - 10)
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 8 - 10)
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 8 - 10)
- (1 - 2 - 3 - 4 - 6 - 3 - 7 - 9 - 10)

Bài 2. Cho đồ thị dòng dữ liệu như sau:



1. Hãy xác định tất cả các *Complete-path* từ đồ thị này.
2. Các đường đi (0 - 1 - 2 - 3), (1 - 2 - 3 - 2 - 4), (3 - 2 - 4), (2 - 3 - 2) và (0 - 1 - 4) có phải là các *simple-paths* không? Giải thích?
3. Các đường đi (2 - 3 - 2), (2 - 3) và (3 - 2 - 4) có phải là các *loop-free-paths* không? Giải thích?

Bài 3. Cho đồ thị dòng dữ liệu như sau:



1. Hãy xác định tất cả các *Def-clear-path* của các biến x và y.
2. Hãy xác định tất cả các *du-paths* của các biến x và y.
3. Dựa vào các chuẩn của kiểm thử dòng dữ liệu hãy xác định tất cả các *All-p-uses/Some-c-uses* và *All-c-uses/Some-p-uses*.

Bài 4. Cho đoạn lệnh như sau:

```

1 #define MAX=50
2 int intValue=0;
3 int iCount=0;
4 for(int i=0;i<MAX;i++){
5     if(i%2==0){
6         intValue = intValue+i;
7     }
8     else {
9         intValue = intValue*i;
10    } //end if-else
11 } //end for
  
```

1. Biến i trong vòng lặp for có ảnh hưởng đến giá trị của biến intValue không? Tại sao?
2. Hãy xác định các câu lệnh ảnh hưởng đến giá trị của biến intValue tại các câu lệnh 6 và 8.

Bài 5. Cho đoạn lệnh như sau:

```
int returnSumArray(int intArr[], int size){  
    int intCount, intSum;  
    intCount = 10, intSum = 0;  
    while(intCount < size){  
        intSum = intSum + intArr[intCount];  
        intCount++;  
    }//end while  
    return intSum;  
}
```

1. Hãy xây dựng hàm phân mảnh của hàm này.
2. Xây dựng đồ thị dòng dữ liệu và xác định các định nghĩa (def) và sử dụng (use) của tất cả các biến trong chương trình.